



AMRITA
VISHWA VIDYAPEETHAM

School of Engineering
Chennai Campus

23CSE211- DESIGN AND ANALYSIS OF ALGORITHMS

AMRITA VISHWA VIDYAPEETHAM

CHENNAI CAMPUS

Name: SEETHAKA.LAHARI SAI

Roll No: CH.SC.U4CSE24143

Class : CSE-B

Year: 2025-2026

WEEK-8

Program-1 : Write a C program to implement Huffman coding that takes a string as input from the user, calculates the frequency of each character, and generates Huffman codes for all unique characters present in the string. Display the character, its frequency, and the corresponding Huffman code in a tabular format.

Consider the following input string representing a dataset label:

"DATAANALYTICSANDINTELLENCELABORATORY"

Code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_TREE_HT 100
#define MAX_CHAR 256
struct MinHeapNode {
    char data;
    unsigned freq;
    struct MinHeapNode *left, *right;
};
struct MinHeap {
    unsigned size;
    unsigned capacity;
    struct MinHeapNode** array;
};
struct MinHeapNode* newNode(char data, unsigned freq) {
    struct MinHeapNode* temp = (struct MinHeapNode*)malloc(sizeof(struct MinHeapNode));
    temp->left = temp->right = NULL;
    temp->data = data;
    temp->freq = freq;
    return temp;
}
struct MinHeap* createMinHeap(unsigned capacity) {
    struct MinHeap* minHeap = (struct MinHeap*)malloc(sizeof(struct MinHeap));
    minHeap->size = 0;
    minHeap->capacity = capacity;
    minHeap->array = (struct MinHeapNode**)malloc(minHeap->capacity * sizeof(struct MinHeapNode));
    return minHeap;
}
void swapMinHeapNode(struct MinHeapNode** a, struct MinHeapNode** b) {
    struct MinHeapNode* t = *a;
    *a = *b;
    *b = t;
}
```

```

void minHeapify(struct MinHeap* minHeap, int idx) {
    int smallest = idx;
    int left = 2 * idx + 1;
    int right = 2 * idx + 2;

    if (left < minHeap->size && minHeap->array[left]->freq < minHeap->array[smallest]->freq)
        smallest = left;

    if (right < minHeap->size && minHeap->array[right]->freq < minHeap->array[smallest]->freq)
        smallest = right;

    if (smallest != idx) {
        swapMinHeapNode(&minHeap->array[smallest], &minHeap->array[idx]);
        minHeapify(minHeap, smallest);
    }
}

struct MinHeapNode* extractMin(struct MinHeap* minHeap) {
    struct MinHeapNode* temp = minHeap->array[0];
    minHeap->array[0] = minHeap->array[minHeap->size - 1];
    --minHeap->size;
    minHeapify(minHeap, 0);
    return temp;
}

void insertMinHeap(struct MinHeap* minHeap, struct MinHeapNode* minHeapNode) {
    ++minHeap->size;
    int i = minHeap->size - 1;
    while (i && minHeapNode->freq < minHeap->array[(i - 1) / 2]->freq) {
        minHeap->array[i] = minHeap->array[(i - 1) / 2];
        i = (i - 1) / 2;
    }
    minHeap->array[i] = minHeapNode;
}

```

```

void buildMinHeap(struct MinHeap* minHeap) {
    int n = minHeap->size - 1;
    for (int i = (n - 1) / 2; i >= 0; --i)
        minHeapify(minHeap, i);
}

int isLeaf(struct MinHeapNode* root) {
    return !(root->left) && !(root->right);
}

void printCodes(struct MinHeapNode* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }
    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }
    if (isLeaf(root)) {
        printf(" %c | %3d | ", root->data, root->freq);
        for (int i = 0; i < top; ++i)
            printf("%d", arr[i]);
        printf("\n");
    }
}

```

```

void calculateSpaceSaving(struct MinHeapNode* root, int arr[], int top, int* totalOriginalBits, int* totalCompressedBits) {
    if (root->left) {
        arr[top] = 0;
        calculateSpaceSaving(root->left, arr, top + 1, totalOriginalBits, totalCompressedBits);
    }
    if (root->right) {
        arr[top] = 1;
        calculateSpaceSaving(root->right, arr, top + 1, totalOriginalBits, totalCompressedBits);
    }
    if (isLeaf(root)) {
        *totalOriginalBits += root->freq * 8; // Assuming 8 bits per character
        *totalCompressedBits += root->freq * top;
    }
}

```

```

void HuffmanCodes(char data[], int freq[], int size) {
    struct MinHeapNode *left, *right, *top;
    struct MinHeap* minHeap = createMinHeap(size);
    for (int i = 0; i < size; ++i)
        minHeap->array[i] = newNode(data[i], freq[i]);
    minHeap->size = size;
    buildMinHeap(minHeap);
    while (minHeap->size != 1) {
        left = extractMin(minHeap);
        right = extractMin(minHeap);
        top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
        insertMinHeap(minHeap, top);
    }
    int arr[MAX_TREE_HT], topIdx = 0;
    printf("\nChar | Freq | Huffman Code\n");
    printf("-----\n");
    struct MinHeapNode* root = extractMin(minHeap);
    printCodes(root, arr, topIdx);
    int totalOriginalBits = 0, totalCompressedBits = 0;
    calculateSpaceSaving(root, arr, 0, &totalOriginalBits, &totalCompressedBits);
    printf("\nSpace Saving Analysis:\n");
    printf("-----\n");
    printf("Original size (8 bits per character): %d bits\n", totalOriginalBits);
    printf("Compressed size (Huffman coding): %d bits\n", totalCompressedBits);
    printf("Space saved: %d bits (%.2f%%)\n",
        totalOriginalBits - totalCompressedBits,
        ((float)(totalOriginalBits - totalCompressedBits) / totalOriginalBits) * 100);
}

```

```

int main() {
    printf("CH.SC.U4CSE24143\n");
    char str[1000];
    int freq[MAX_CHAR] = {0};
    printf("Enter the string: ");
    scanf("%s", str);
    for (int i = 0; str[i] != '\0'; i++) {
        freq[(unsigned char)str[i]]++;
    }
    char unique_chars[MAX_CHAR];
    int unique_freqs[MAX_CHAR];
    int n = 0;
    for (int i = 0; i < MAX_CHAR; i++) {
        if (freq[i] > 0) {
            unique_chars[n] = (char)i;
            unique_freqs[n] = freq[i];
            n++;
        }
    }
    HuffmanCodes(unique_chars, unique_freqs, n);
    return 0;
}

```

Output:

```
lahari@DESKTOP-JIGL8HI:~$ ./huffman
CH.SC.U4CSE24143
Enter the string: Lahari
```

Char	Freq	Huffman Code
a	2	0
L	1	100
r	1	101
i	1	110
h	1	111

Space Saving Analysis:

Original size (8 bits per character): 48 bits
Compressed size (Huffman coding): 14 bits
Space saved: 34 bits (70.83%)

Time Complexity: $O(N + M \log M)$:

$O(n + m \log m)$ where N is the input string length and M is the number of unique characters – the frequency counting takes $O(N)$ while building the Huffman tree with m nodes takes $O(M \log M)$ due to repeated extractMin and insert operations in the min-heap.

Space Complexity: $O(N + M)$:

$O(N + M)$ where N stores the input string and M stores the Huffman tree nodes and related data structures – since M is bounded by 256 (ASCII characters), this effectively becomes $O(N)$ for practical purposes.

WORKING

HUFFMAN CODING

Consider the following input string representing a chatbot label.

"DATA ANALYTICS AND INTELLIGENCE LABORATORY"

Apply huffman coding algorithm to compress this string.
Also construct a huffman tree by showing step by step merging.

Step 1 Frequency Analysis

Letter Count

A	7
L	4
N	4
T	4
E	3
I	3
C	2
D	2
O	2
R	2
Y	2
B	1
G	1
S	1

Step 2

Arrange them in ascending order:

B(1) - G(1) - S(1) - C(2) - D(2) - O(2) - R(2) - Y(2) - E(3) - I(3) -
L(4) - N(4) - T(4) - A(7)

Step 3

Apply huffman algorithm to build the huffman tree

Initially

$B[1] = G[1] = S[1] = C[2] = O[2] = o[2] = R[2] = Y[2] = E[3]$,
 $I[3] = L[4] = N[4] = T[4] = A[7]$

St-1: Combine $B[1]$ and $G[1]$ to get $BG[2]$
 $S[1] = C[2] = O[2] = o[2] = R[2] = Y[2] = BG[2] = E[3] = I[3]$,
 $L[4] = N[4] = T[4] = A[7]$

St-2: Combine $S[1]$ and $C[2]$ to get $SC[3]$
 $O[2] = o[2] = R[2] = Y[2] = BG[2] = E[3] = I[3] = SC[3]$,
 $L[4] = N[4] = T[4] = A[7]$

St-3: Combine $O[2]$ and $o[2]$ to get $OO[4]$
 $R[2] = Y[2] = BG[2] = E[3] = I[3] = SC[3] = L[4] = N[4] = T[4]$,
 $OO[4] = A[7]$

St-4: Combine $R[2]$ and $Y[2]$ to get $RY[4]$
 $BG[2] = E[3] = I[3] = SC[3] = L[4] = N[4] = T[4] = OO[4] = RY[4]$,
 $= A[7]$

St-5: Combine $BG[2]$ and $E[3]$ to get $BGE[5]$
 $I[3] = SC[3] = L[4] = N[4] = T[4] = OO[4] = RY[4] = BGE[5] = A[7]$

St-6: Combine $I[3]$ and $SC[3]$ to get $ISC[6]$
 $L[4] = N[4] = T[4] = OO[4] = RY[4] = BGE[5] = ISC[6] = A[7]$

St-7: Combine $L[4]$ and $N[4]$ to get $LN[8]$
 $T[4] = OO[4] = RY[4] = BGE[5] = ISC[6] = A[7] = LN[8]$

St-8: Combine $T[4]$ and $OO[4]$ to get $TOO[8]$
 $RY[4] = BGE[5] = ISC[6] = A[7] = LN[8] = TOO[8]$

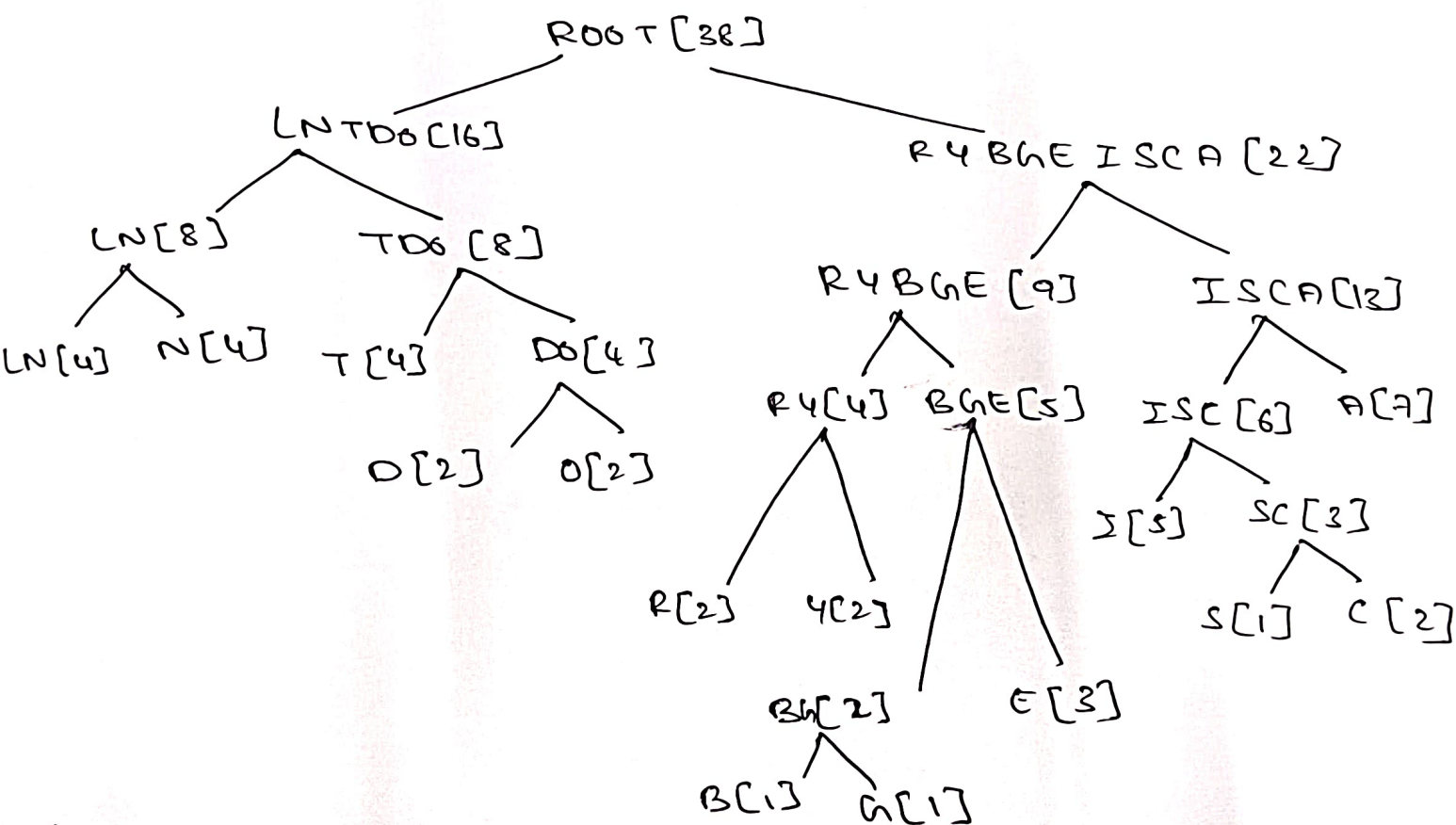
Step-9 Combine $RY[4]$ and $BGE[5]$ to get $RYBGE[9]$
 $ISC[6] - A[7] - LN[8] - TDO[8] - RYBGE[9]$

Step-10 Combine $ISC[6]$ and $A[7]$ to get $ISCA[13]$
 $LN[8] - TDO[8] - RYBGE[9] - ISCA[13]$

Step-11 Combine $LN[8]$ and $TDO[8]$ to get $LNTDO[16]$
 $RYBGE[9] - ISCA[13] - LNTDO[16]$

Step-12 Combine $RYBGE[9]$ and $ISCA[13]$ to get $RYBGEISCA[22]$
 $LNTDO[16] - RYBGEISCA[22]$

Combine $LNTDO[16]$ and $RYBGEISCA[22]$ to get
 $"LNTDORYBGEISCA[38]"$



Huffman codes:

$A = 111$; $L = 000$; $N = 001$; $T = 010$; $E = 1011$;
 $I = 1100$; $C = 11011$; $O = 0110$; $O = 0111$; $R = 1000$;
 $Y = 1001$; $B = 10100$; $G = 10101$; $S = 11010$;